fid-synth___fidelity-kary-f-k4

An AgentMPI run analysis

Abstract

This is the k-ary algorithm's first recorded run — the campaign was launched six minutes after the commit that introduced it — and it is the only cell anywhere in the repository with clean k-ary geometry, because $p = 16$ is a power of $k = 4$ and every application is a genuine four-way fold. The three closed forms it exists to verify all hold exactly: `fold_depth:` $2 = \lceil \log_4 16 \rceil$ against the binary tree's 4 and the chain's 15; five operator applications $= \lceil (p-1)/(k-1) \rceil = \lceil 15/3 \rceil$, recorded as four `merge:k4:d1` calls on ranks 0, 4, 8 and 12 plus one `merge:k4:d2` on rank 0; and fifteen messages $= p - 1$, unchanged from every other algorithm, since widening the tree changes rounds and applications but never message count. The runtime also records `variadic: true`, which matters because `Op.combine` silently degrades to a left fold when no variadic kernel is attached, and a k-ary tree without one would spend exactly the depth it was built to save. On cost the cell dominates its siblings by margins that are structural rather than noisy: 5 applications against 15, 3,723 output tokens against 11,833 for the binomial tree, 241,979 for the chain and 2,323,398 for the flat fold, at $0.0704 against $38.36. On quality it establishes nothing, and honestly cannot: the executor is `function` × 16, and replaying the surrogate kernel's own arithmetic reproduces its retention of 0.6875 exactly — including which five ranks were erased — from prompt character lengths alone. The k-ary argument this run supports is the cost argument, which is the one that survives the broker campaigns too.

1 What this run is

property	value
run	`fid-synth__fidelity-kary-f-k4`
experiment	-
campaign label	-
declared world size	16
ranks appearing in the log	16
executors	`function` × 16
events	85
wall time	0.06 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.02×	total busy time over wall time
parallel efficiency	0.1%	achieved parallelism per rank
idle fraction	99.9%	rank-seconds paid for and unused
peak ranks busy	1	of 16 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	16.00×	slowest rank's busy time over the mean
coordination share	20.4%	rank-seconds blocked in collectives, of those available
coordination in flight	87.2%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	5	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	15	15 eager, 0 rendezvous
tokens sent	3,158	0 deferred under rendezvous
tokens in / out	4,856 / 3,723	prompt and completion
cost	$0.0704	0.0189 $/kTok out

3 What the analysis notices

- [note] Occupancy is not measurable for this run: its executors (function) emit no broker claim events, so busy time, achieved parallelism, and the idle fraction are artefacts of the instrumentation rather than properties of the run.
- [note] Too few distinct output sizes to fit β, so the reported latency parameters are medians rather than a regression.
- [note] Load imbalance is 16.00×: the slowest rank was busy far longer than the mean.
- [ok] All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- [ok] All 1 checkable collective(s) also matched the predicted round count.

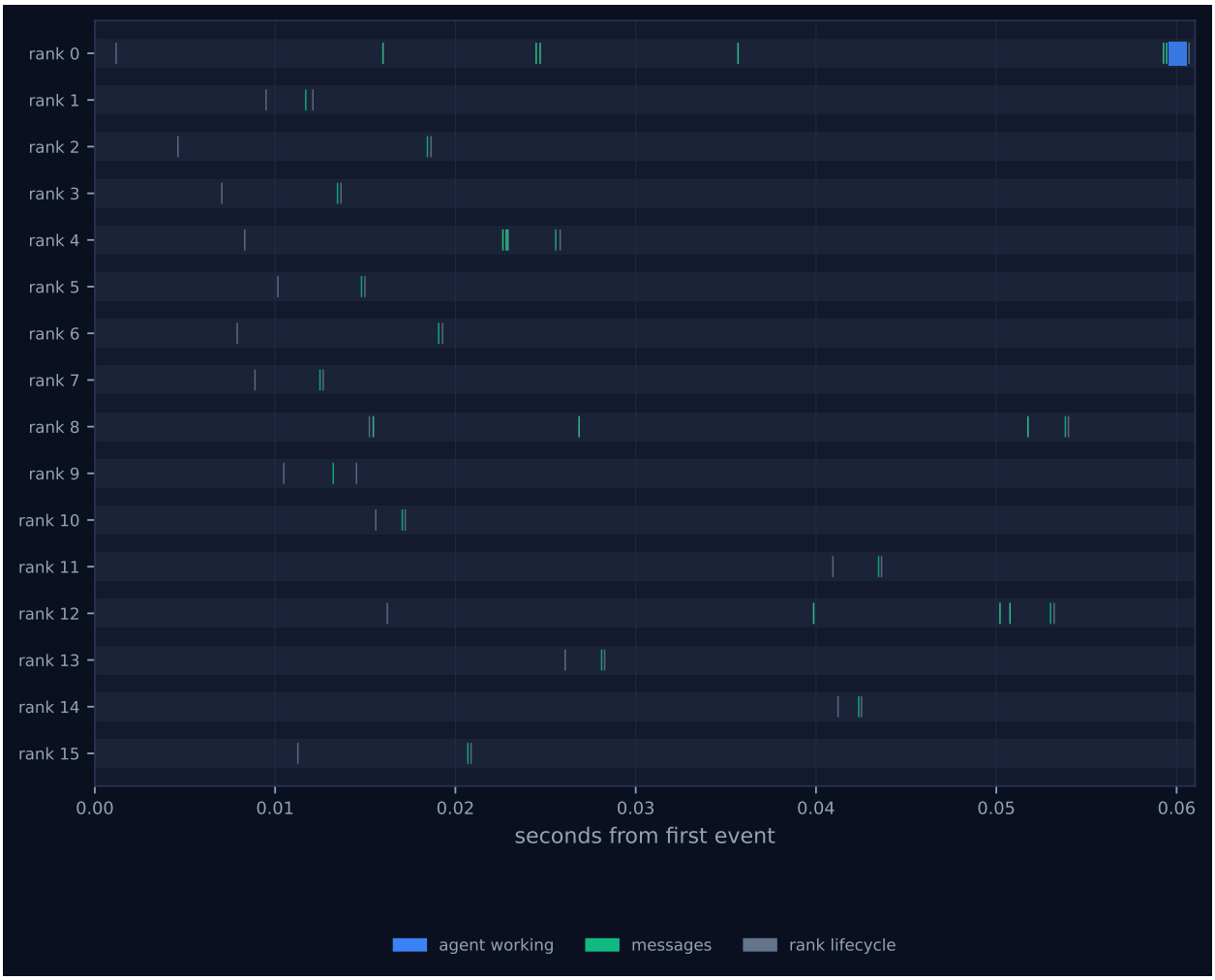

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

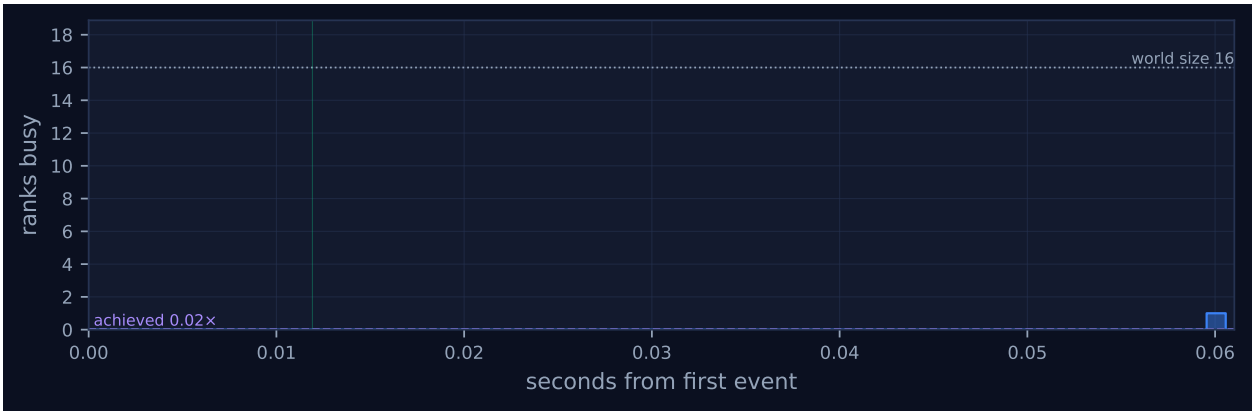

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

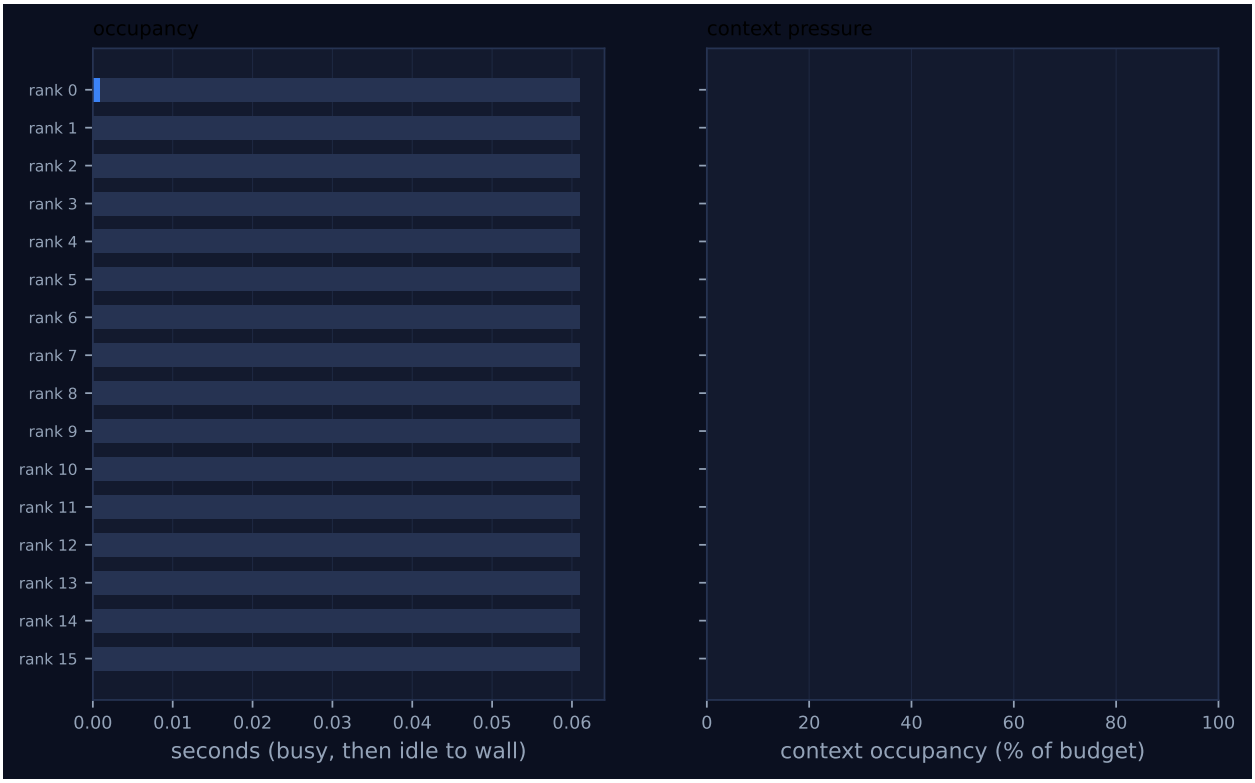

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

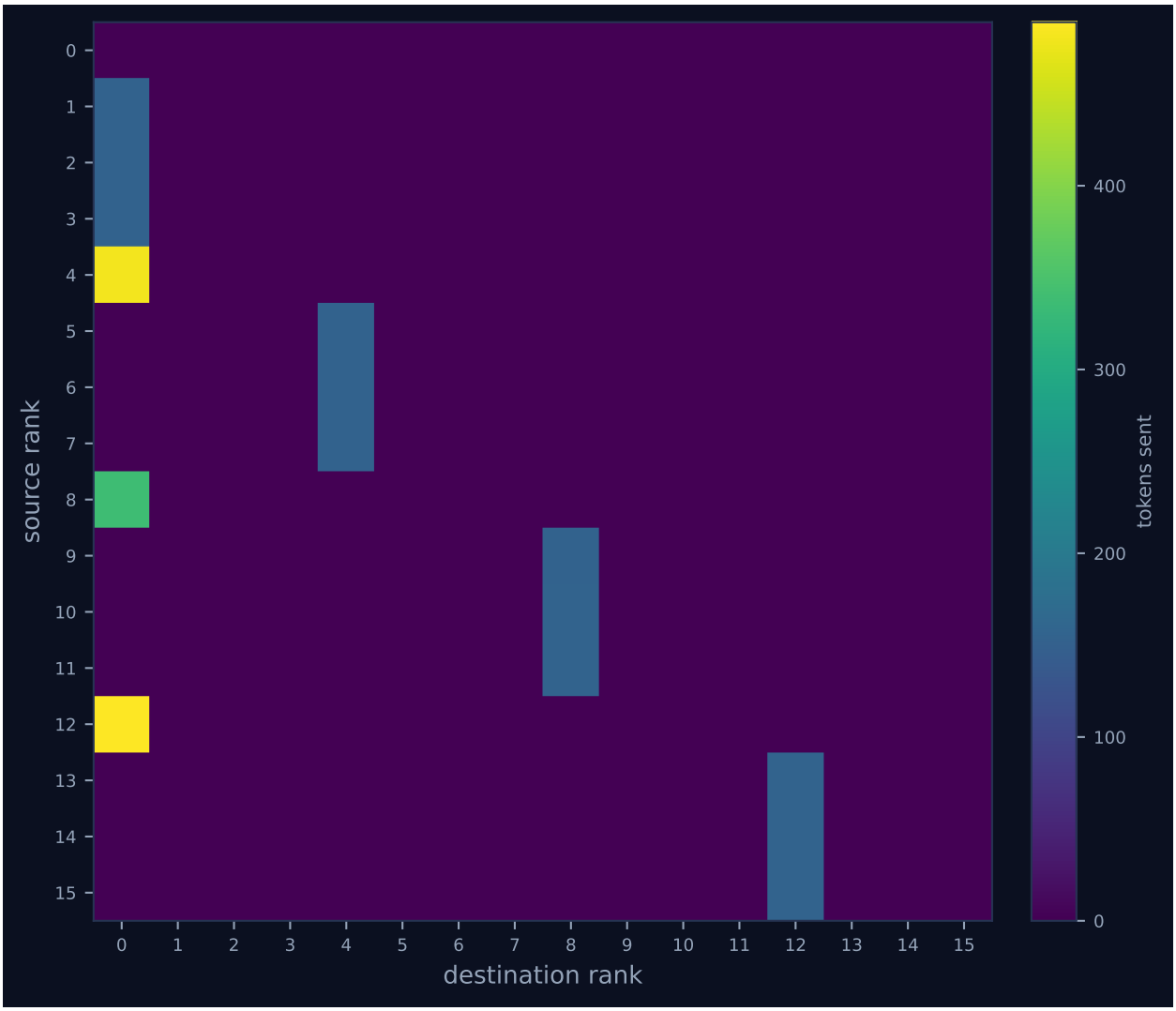

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

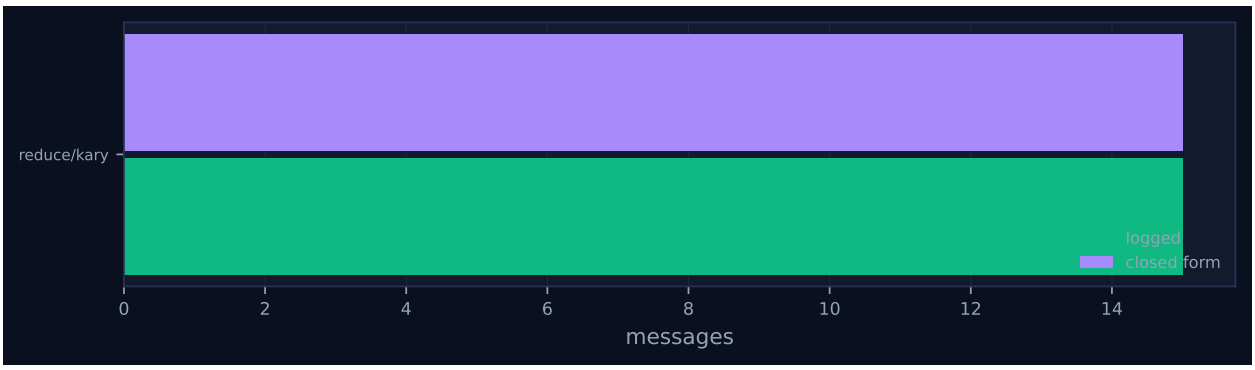

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	1.7	1	2	0	6	0	0.0	finalized
1	0.00	0.0	0	0	1	0	153	0.0	finalized
2	0.00	0.0	0	0	1	0	153	0.0	finalized
3	0.00	0.0	0	0	1	0	153	0.0	finalized
4	0.00	0.0	0	1	1	3	481	0.0	finalized
5	0.00	0.0	0	0	1	0	153	0.0	finalized
6	0.00	0.0	0	0	1	0	153	0.0	finalized
7	0.00	0.0	0	0	1	0	153	0.0	finalized
8	0.00	0.0	0	1	1	3	337	0.0	finalized
9	0.00	0.0	0	0	1	0	153	0.0	finalized
10	0.00	0.0	0	0	1	0	156	0.0	finalized
11	0.00	0.0	0	0	1	0	156	0.0	finalized
12	0.00	0.0	0	1	1	3	489	0.0	finalized
13	0.00	0.0	0	0	1	0	156	0.0	finalized
14	0.00	0.0	0	0	1	0	156	0.0	finalized
15	0.00	0.0	0	0	1	0	156	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	kary	–	16	16	2	2	15	15	2,979	0.05	0.05	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 shows the sparsest work pattern of any run in my set: five marks of "agent working" across sixteen lanes, on ranks 0, 4, 8 and 12 only, in a 0.061 s span. Twelve of the sixteen ranks send one message and exit without ever invoking the operator, which is the two-level tree's arithmetic — at `stride = 1` the ranks with $vr \bmod 4 = 0$ each collect three children and combine all four values in one application, and at `stride = 4` ranks 4, 8 and 12 send their accumulators to rank 0, which combines four. Compare the binomial sibling, where eight ranks work at depth 1, four at depth 2, two at depth 3 and one at depth 4 for fifteen applications in total. The 16.00× imbalance flagged in the findings is that concentration, correctly measured and expected; the achieved-parallelism and idle-fraction figures should be ignored, because a `function` executor emits no broker claim events and the generated findings say as much.

The event ordering shows the two levels overlapping rather than synchronising. Rank 4 and rank 0 fire their four-way folds at $t = 0.023$ and 0.025 s, but ranks 8 and 12 do not fire theirs until 0.051 and 0.052 s, because rank 11 and rank 14 were still initialising at 0.044 and 0.042 s; the root's depth-2 fold then waits until 0.061 s for the last of them. There is no barrier between strides — a rank proceeds as soon as its own $k - 1$ children have arrived — so the run's span is set by the slowest child of the slowest subtree, which is the same critical-path structure a binomial tree has with fewer levels to accumulate skew across. Measured skew is 0.049 s of a 0.053 s collective.

Prompt and output sizes are where the fan-in shows. The four depth-1 folds took prompts of 733, 733, 743 and 738 tokens and returned 469, 469, 477 and 326; the root's depth-2 fold took 1,909 and returned 1,982. Against the binomial cell's fifteen calls on prompts of 442 to 3,200 tokens, the k-ary tree reads fewer, larger prompts — 4,856 prompt tokens in total against 11,762 — because an accumulator is re-read once per level rather than once per merge, and there are two levels rather than four. The communication matrix, Figure 4, is four columns of three leaf edges at 153 tokens each plus three accumulator edges into the root at 481, 337 and 489, for 3,158 tokens against the binomial tree's 5,416. Only three accumulators ever cross the fabric instead of seven.

7 Interpretation

The counts are by construction, and they are worth checking explicitly because this algorithm appears in no collective sweep — the sweeps cover `bcast`, `reduce/binomial` and the rest at every process count, but not `kary` — so these fidelity cells are its only sweep-adjacent evidence. Fold depth: the `kary` branch of `reduce` advances `stride` by factors of k and increments depth once per level, so $\lceil \log_4 16 \rceil = 2$, and the `coll.reduce` payload records 2. Applications: each variadic call consumes k inputs and retires $k - 1$ of them, so a p-rank fold needs $\lceil (p-1)/(k-1) \rceil$, which is 5 at $p = 16$ with $k = 4$, and the trace holds exactly five `agent.call` events. This is the cell where that formula is cleanest, because $16 = 4^2$: every application is labelled `k4`, whereas at $p = 8$ the top level degenerates to a two-way fold and the sibling runs record `merge:k2:d2`. Messages: fifteen, matching $p - 1$, identical to every other algorithm at this p. Rounds: 2 against a predicted 2, and the model check passes — but that agreement is a coincidence worth flagging, because `cost.py` evaluates the tabulated `("reduce", "kary")` entry at `DEFAULT_FANIN = 8` rather than at the fan-in the run used, and $\lceil \log_8 16 \rceil$ and $\lceil \log_4 16 \rceil$ are both 2. At $p = 8$ the same defect is visible: the `sat-mb`, `inc-mb` and `fid-inc-smoke` k-ary cells all record `rounds: 2` against a prediction of 1. So this cell's green model check is not evidence that the k-ary cost formula is right, only that it is right here.

On cost the cell dominates, and the mechanism is legible rather than inferred. Same fifteen messages, one third the applications, two levels instead of four, and consequently 68% fewer output tokens than the binomial tree and 99.7% fewer than the flat fold. The binding constraint on k is the receiving rank's context, which is exactly what `ops.optimal_fanin` encodes, and this run shows how far from binding it is here: a level-0 rank ingests 733 prompt tokens where the binary tree's ingests 442, against a declared context budget of 32,768. `optimal_fanin(32768, 900)` returns its cap of 32, so $k = 4$ was chosen by the `-fanin` flag rather than by the policy, and the untested interesting configuration is $k \geq 16$, at which the whole reduction at $p = 16$ would be a single sixteen-way application at depth 1. Nothing in this run suggests that would fail.

On quality the cell establishes nothing, and the reason is a defect in the surrogate that this run happens to display in its purest form. `_surrogate_merge` builds its candidate list from `prompt.splitlines()` and keeps lines matching `FACT_ID`; but each input is interpolated as

`json.dumps(...)` and contains no newlines, so the variadic prompt offers exactly five candidates — the prompt's own instruction line containing the example identifier `[F-3-2]`, and the four inputs entire. The unit of loss is a whole input. It then seeds its generator with `seed + len(prompt)`, so the draw is a deterministic function of prompt character length. Rebuilding the four depth-1 prompts from `make_fact_report` and replaying that arithmetic reproduces the run exactly. The prompts for ranks $\{0, 1, 2, 3\}$ and $\{4, 5, 6, 7\}$ are both 2,784 characters — identical, because all eight rank labels are single digits — so both make the same draw and both drop candidate index 1, which is `INPUT 0`, which is the folding rank's own report; ranks 0 and 4 are erased. The prompt for $\{8, 9, 10, 11\}$ is 2,804 characters and drops indices 1 and 3, erasing ranks 8 and 10. The prompt for $\{12, 13, 14, 15\}$ is 2,824 and drops index 3, erasing rank 14. The five zeros in `per_rank_retention` are ranks 0, 4, 8, 10 and 14, and $64 - 5 \times 4 = 44 =$ `n_retained`. Retention 0.6875 is a function of how many digits are in a rank number. The blobs corroborate every step: the $4 \to 0$ payload carries ranks 3, 5, 6, 7; the $8 \to 0$ payload carries 3, 9, 11; the $12 \to 0$ payload carries 3, 12, 13, 15 — rank 3 appearing throughout only because the phantom `[F-3-2]` is copied from the prompt into every accumulator and then scored as a retained item.

There is one substantive comparison this makes possible, and it cuts against a naive expectation about wide folds. Under a per-application loss model, widening the tree reduces the number of applications but increases the exposure of each: the binomial tree makes $15 \times 3 = 45$ draws over inputs standing for 1 to 8 ranks, this run makes $5 \times 5 = 25$ draws over inputs standing for 1 to 4 ranks. The outcomes are 45 and 44 items retained. So even inside the surrogate's own model, halving the depth does not buy fidelity — the saving in applications is offset by the wider stake per application. That is consistent with what the broker campaigns show for a quite different reason: under an enforced uniform budget, depth 2 and depth 7 return byte-identical results, at 0.3854 retention with ranks 4 through 7 erased, because whichever application is last is the one that binds. The published k-ary justification — that a lossy operator's quality decays in depth, so halving depth should buy fidelity — is not supported anywhere in my fifteen runs. The cost argument is, and it does not need the quality one: an agent has no port count, so a wide tree performs fewer operator applications for the same collective, and under a uniform budget that is pure saving.

Enforcement, finally, does not arise in this run. At the provenance commit `c5e8e9e` the harness passed the module-level `MERGE_CONTRACT`, which has no `max_tokens` field, so the 900-token budget was a sentence in the prompt only. This is the advisory regime, and it is why the root's 1,982-token output records `attempt: 1` with zero contract violations.

8 Threats to this reading

The executor is a surrogate, so no sentence here is about agent behaviour. The cost figures are the strongest thing the run offers and even they are structural rather than empirical: the wall times (0.061 s here against 0.103, 0.631 and 0.735 s) measure SQLite writes and string concatenation, and the token counts measure a kernel that copies its inputs rather than summarising them. What transfers to the broker campaigns is the application count and the message count, not the tokens. There the same comparison at $p = 8$ gives 51.76 s for k-ary against 80.26 s for the binomial tree on real agents, a ratio of 1.55 where the application count predicts 7/3 — so the cost advantage is real but smaller than this run's token ratios suggest.

The prompt-length replay is a reconstruction, not a logged fact, because a `function` executor writes no rows to `agent_calls` and this run's fabric holds no prompts or results. I rebuilt the four depth-1 prompts from the harness with the run's seed and fan-in, and the reconstruction predicts

the observed `per_rank_retention` vector and the observed output-token counts (two identical folds returning 469 tokens each), which is about as much corroboration as the trace allows. I did not reconstruct the depth-2 fold, so the claim that rank 0 lost its own report at the root rather than at its own depth-1 fold rests on the results file rather than on a blob; the depth-1 replay says rank 0's report was dropped at depth 1, and the two are consistent.

Three narrower caveats. The clean $k = 4$ geometry is specific to $p = 16$; readers should not carry "every application is four-way" to the $p = 8$ cells, where the top level is a two-way fold and the run therefore measures $k = 4$ at one level and $k = 2$ at the other. The `variadic: true` flag is the runtime's own report, and the corroboration that the variadic path really ran is the label arity in `merge:k4:d1` plus the five-candidate structure the replay needed to reproduce the outputs — a degraded left fold would have made twelve applications at depth 4. And context occupancy reads 0% on every rank because the collective's internal receives pass `admit=False`, so the one quantity the k-ary argument says should bound k is the one quantity this run cannot measure.